CSC 4520/6520
Design & Analysis of Algorithms

CHAPTER 3: EXHAUSTIVE SEARCH ALGORITHMS

Abdullah Bal, PhD

Objectives

Recap

Brute Force

Selection Sort

Bubble Sort

String Matching

Closest pair problem

Exhaustive Search

Travelling Salesman Problem

Knapsack Problem

Assignment problem

Exhaustive Search

- ***Exhaustive search*** is simply a brute-force approach to combinatorial problems.
- Many important problems require finding an element with a special property in a domain that grows exponentially (or faster) with an instance size.
- Typically, such problems arise in situations that involve—explicitly or implicitly—combinatorial objects such as permutations, combinations, and subsets of a given set.

Exhaustive Search

- Many such problems are optimization problems: they ask to find an element that maximizes or minimizes some desired characteristic such as a path length or an assignment cost.
- It suggests generating each and every element of the problem domain, selecting those of them that satisfy all the constraints, and then finding a desired element (e.g., the one that optimizes some objective function).
- Although the idea of exhaustive search is quite straightforward, its implementation typically requires an algorithm for generating certain combinatorial objects.

Exhaustive Search

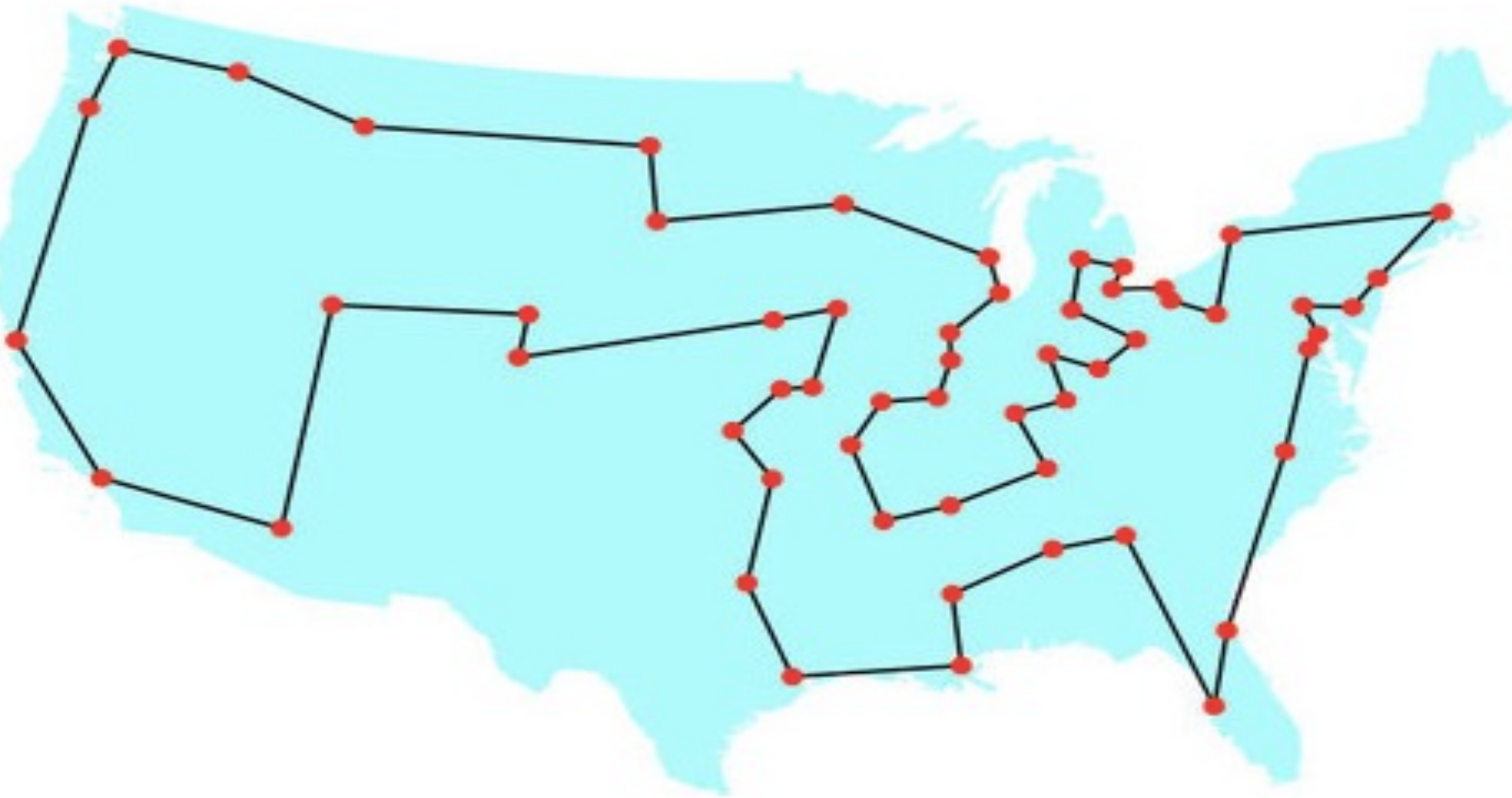
- We illustrate exhaustive search by applying it to three important problems:
  - The traveling salesman problem
  - The knapsack problem
  - The assignment problem.

Exhaustive Search

Method:

- Generate a list of all potential solutions to the problem in a systematic manner.
- Evaluate potential solutions one by one, disqualifying infeasible ones and, for an optimization problem, keeping track of the best one found so far
- When search ends, present the solution(s) found

Traveling Salesman Problem (TSP)



Example 1: Traveling Salesman Problem (TSP)

- The ***traveling salesman problem (TSP)*** has been intriguing researchers for the last 150 years by its seemingly simple formulation, important applications, and interesting connections to other combinatorial problems.
- The problem asks to find the shortest tour through a given set of n cities that visits each city exactly once before returning to the city where it started.
- The problem can be conveniently modeled by a weighted graph, with the graph's vertices representing the cities and the edge weights specifying the distances.

Example 1: Traveling Salesman Problem (TSP)

- The problem can be stated as the problem of finding the shortest ***Hamiltonian circuit*** of the graph.
- A Hamiltonian circuit is defined as a cycle that passes through all the vertices of the graph exactly once.
- It is named after the Irish mathematician Sir William Rowan Hamilton (1805–1865), who became interested in such cycles as an application of his algebraic discoveries.

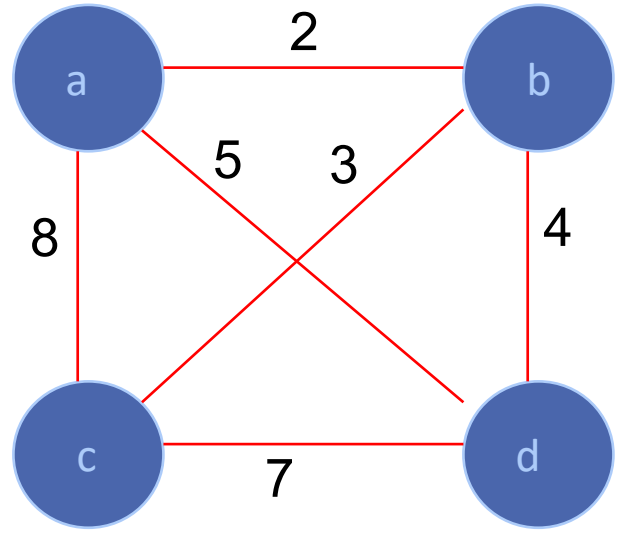
Example 1: Traveling Salesman Problem (TSP)

- A Hamiltonian circuit can also be defined as a sequence of n + 1 adjacent vertices.

$$v_{i0}, v_{i1}, \ldots, v_{in-1}, v_{i0}$$

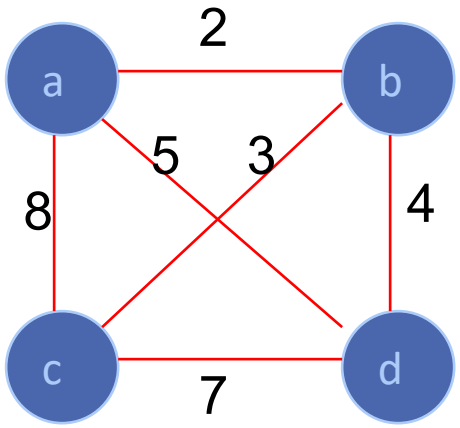
where the first vertex of the sequence is the same as the last one and all the other n − 1 vertices are distinct.

- All circuits start and end at one particular vertex.
- We can get all the tours by generating all the permutations of n − 1 intermediate cities, compute the tour lengths, and find the shortest among them.



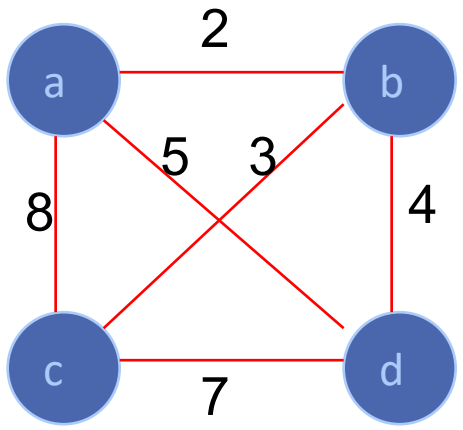
Example 1: Traveling Salesman Problem (TSP)

- An inspection of the figure reveals three pairs of tours that differ only by their direction.
- Hence, we could cut the number of vertex permutations by half.
- We could, for example, choose any two intermediate vertices, say, b and c, and then consider only permutations in which b precedes c.
- The total number of permutations needed is still ½ . (n – 1)!, which makes the exhaustive-search approach impractical for all but very small values of n.
- Time complexity of the traveling salesman problem is Θ(n!).
- We are not limited our investigation to the circuits starting at the same vertex.
- The number of permutations would have been even larger, by a factor of n.

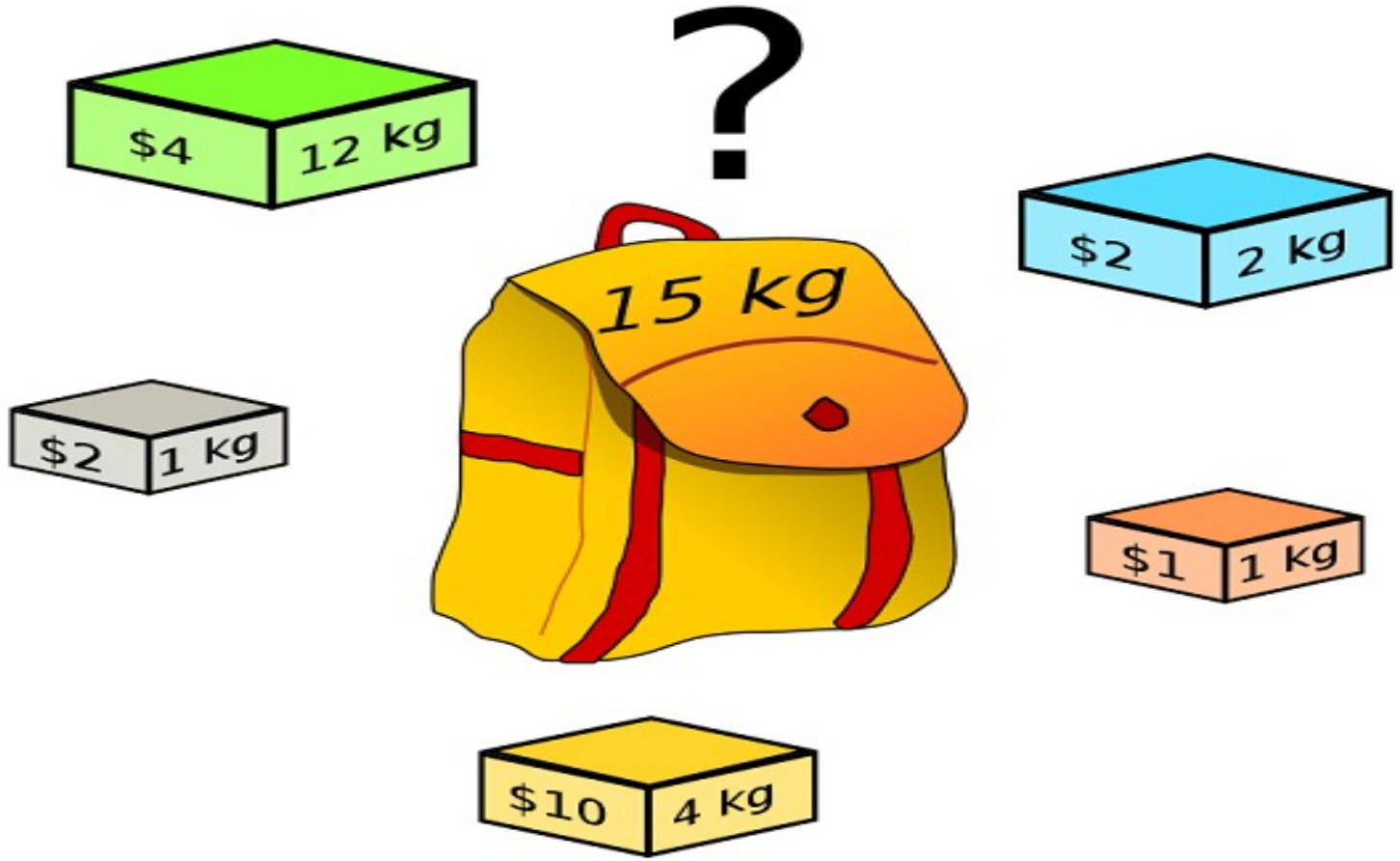


Example 1: TSP by Exhaustive Search

Tour	Cost
a→b→c→d→a	**2+3+7+5 = 17**
a→b→d→c→a	2+4+7+8 = 21
a→c→b→d→a	8+3+4+5 = 20
a→c→d→b→a	8+7+4+2 = 21
a→d→b→c→a	5+4+3+8 = 20
a→d→c→b→a	**5+7+3+2 = 17**



Knapsack Problem



Example 2: Knapsack Problem

- Given n items of known

  weights $w_1, w_2, \ldots, w_n$

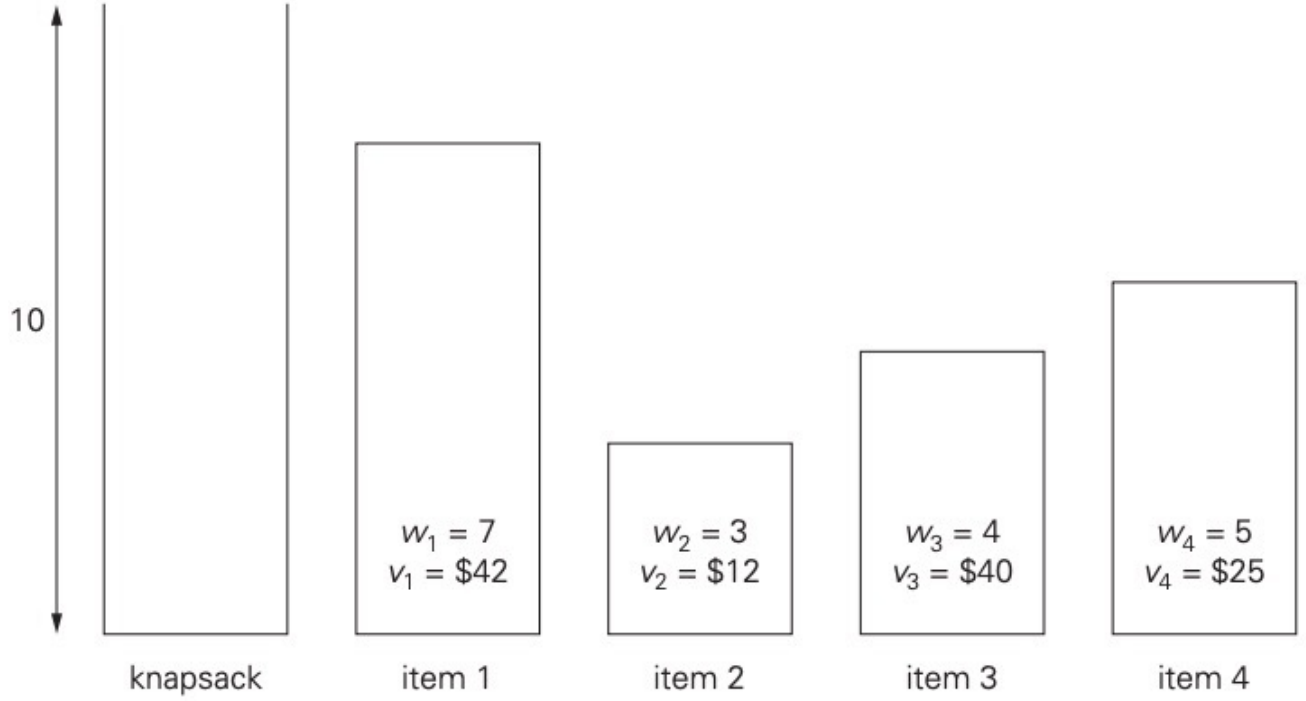
  values $v_1, v_2, \ldots, v_n$

  a knapsack of capacity W,

find the most valuable subset of the items that fit into the knapsack.

- Think about a transport plane that has to deliver the most valuable set of items to a remote location without exceeding the plane's capacity.

Example 2: Knapsack Problem



Subset	Total weight	Total value
∅	0	$ 0
{1}	7	$42
{2}	3	$12
{3}	4	$40
{4}	5	$25
{1, 2}	10	$54
{1, 3}	11	not feasible
{1, 4}	12	not feasible
{2, 3}	7	$52
{2, 4}	8	$37
{3, 4}	**9**	**$65**
{1, 2, 3}	14	not feasible
{1, 2, 4}	15	not feasible
{1, 3, 4}	16	not feasible
{2, 3, 4}	12	not feasible
{1, 2, 3, 4}	19	not feasible

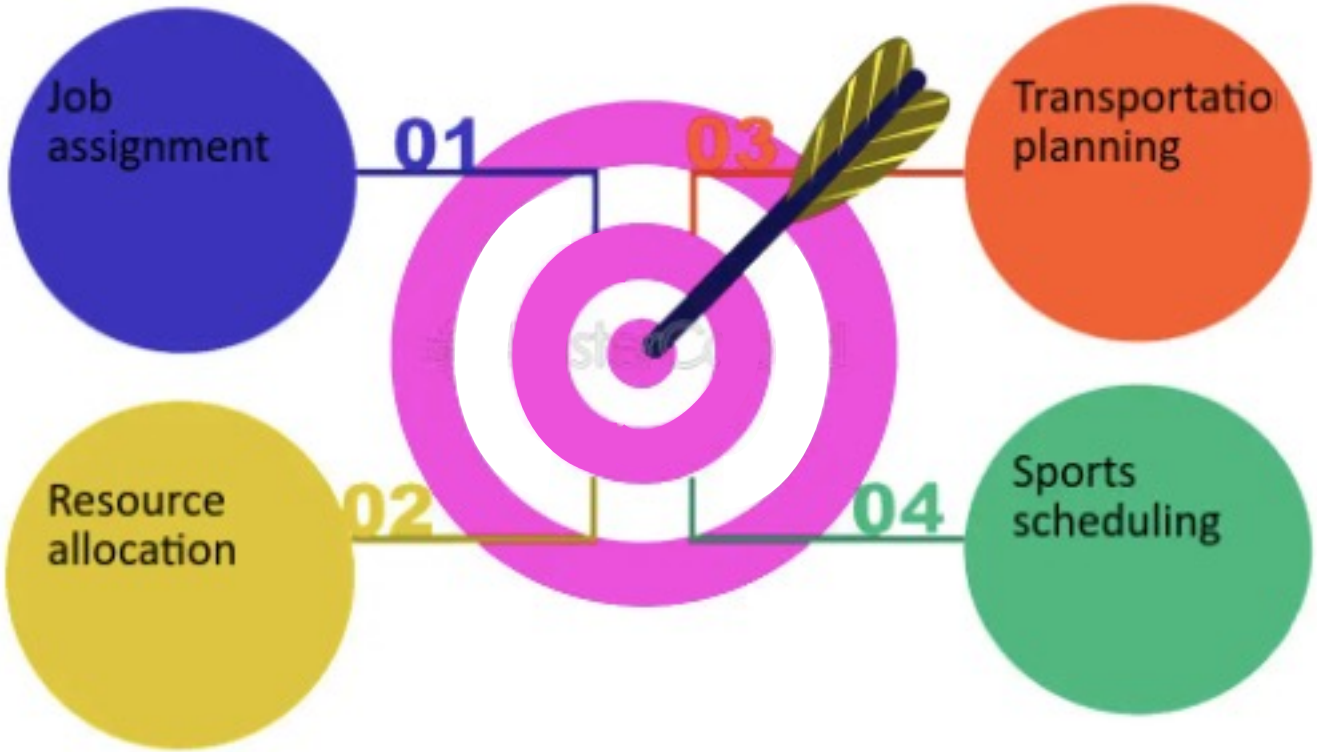
Example 2: Knapsack Problem

- The exhaustive-search approach to this problem leads to generating all the subsets of the set of n items given, computing the total weight of each subset in order to identify feasible subsets (i.e., the ones with the total weight not exceeding the knapsack capacity),
- Finding a subset of the largest value among them.
- Since the number of subsets of an n-element set is 2^n, the exhaustive search leads to a $\Omega(2^n)$ algorithm, no matter how efficiently individual subsets are generated.

Evaluation of the Exhaustive Search

- For both the traveling salesman and knapsack problems, exhaustive search leads to algorithms that are extremely inefficient on every input.
- In fact, these two problems are the best-known examples of No polynomial-time algorithm is known for *NP*- hard problem
- More-sophisticated approaches—backtracking and branch-and-bound enable us to solve some but not all instances of these and similar problems in less than exponential time.

The Assignment Problem



Example 3: The Assignment Problem

- There are n people who need to be assigned to n jobs, one person per job. The cost of assigning person i to job j is C[i,j].
- Find an assignment that minimizes the total cost.

	Job 0	Job 1	Job 2	Job 3
Person 0	9	2	7	8
Person 1	6	4	3	7
Person 2	5	8	1	8
Person 3	7	6	9	4

- **Algorithmic Plan:** Generate all legitimate assignments, compute their costs and select the cheapest one.
- How many assignments are there?

Example 3: The Assignment Problem

- Pose the problem as the one about a cost matrix:

(first person, first job)

	Job 0	Job 1	Job 2	Job 3
Person 0	9	2	7	8
Person 1	6	4	3	7
Person 2	5	8	1	8
Person 3	7	6	9	4

$$C = \begin{bmatrix} 9 & 2 & 7 & 8 \\ 6 & 4 & 3 & 7 \\ 5 & 8 & 1 & 8 \\ 7 & 6 & 9 & 4 \end{bmatrix}$$

<1, 2, 3, 4> cost = 9 + 4 + 1 + 4 = 18
<1, 2, 4, 3> cost = 9 + 4 + 8 + 9 = 30
<1, 3, 2, 4> cost = 9 + 3 + 8 + 4 = 24
<1, 3, 4, 2> cost = 9 + 3 + 8 + 6 = 26
<1, 4, 2, 3> cost = 9 + 7 + 8 + 9 = 33
<1, 4, 3, 2> cost = 9 + 7 + 1 + 6 = 23

etc.

- Since the number of permutations to be considered for the general case of the assignment problem is n!, exhaustive search is impractical for all but very small instances of the problem.

Example 3: The Assignment Problem

- We can describe feasible solutions to the assignment problem as n-tuples

$$\langle j_1, \ldots, j_n \rangle$$

in which the i^{th} component, i = 1, . . . , n, indicates the column of the element selected in the i^{th} row (i.e., the job number assigned to the ith person).

- For example, for the cost matrix above, ⟨2, 3, 4, 1⟩ indicates the assignment of

  Person 1 to Job 2,

  Person 2 to Job 3,

  Person 3 to Job 4,

  Person 4 to Job 1.

Example 3: The Assignment Problem

- The requirements of the assignment problem imply that there is a one-to-one correspondence between feasible assignments and permutations of the first n integers.
- Therefore, the exhaustive-search approach to the assignment problem would require generating all the permutations of integers 1, 2, . . . , n, computing the total cost of each assignment by summing up the corresponding elements of the cost matrix, and finally selecting the one with the smallest sum.

Example 3: The Assignment Problem

- It is easy to see that an instance of the assignment problem is completely specified by its cost matrix C.
- In terms of this matrix, the problem is to select one element in each row of the matrix so that all selected elements are in different columns and the total sum of the selected elements is the smallest possible.
- Note that no obvious strategy for finding a solution works here.
- For example, we cannot select the smallest element in each row, because the smallest elements may happen to be in the same column.
- In fact, the smallest element in the entire matrix need not be a component of an optimal solution.

Final Comments on Exhaustive Search

- Exhaustive-search algorithms run in a realistic amount of time only on very small instances
- In some cases, there are much better alternatives!
  - Euler circuits
  - Shortest paths
  - Minimum spanning tree
- In many cases, exhaustive search or its variation is the only known way to get exact solution

Summary

- Exhaustive Search
- Travelling Salesman Problem
- Knapsack Problem
- Assignment problem

Supportive Materials

- *Introduction to The Design and Analysis of Algorithms (3rd Edition) by Anany Levitin, Pearson.*
- *Introduction to Algorithms*, by T.Cormen, C. Leiserson, R.Rivest and C.Stein, MIT Press, 3rd Edition.